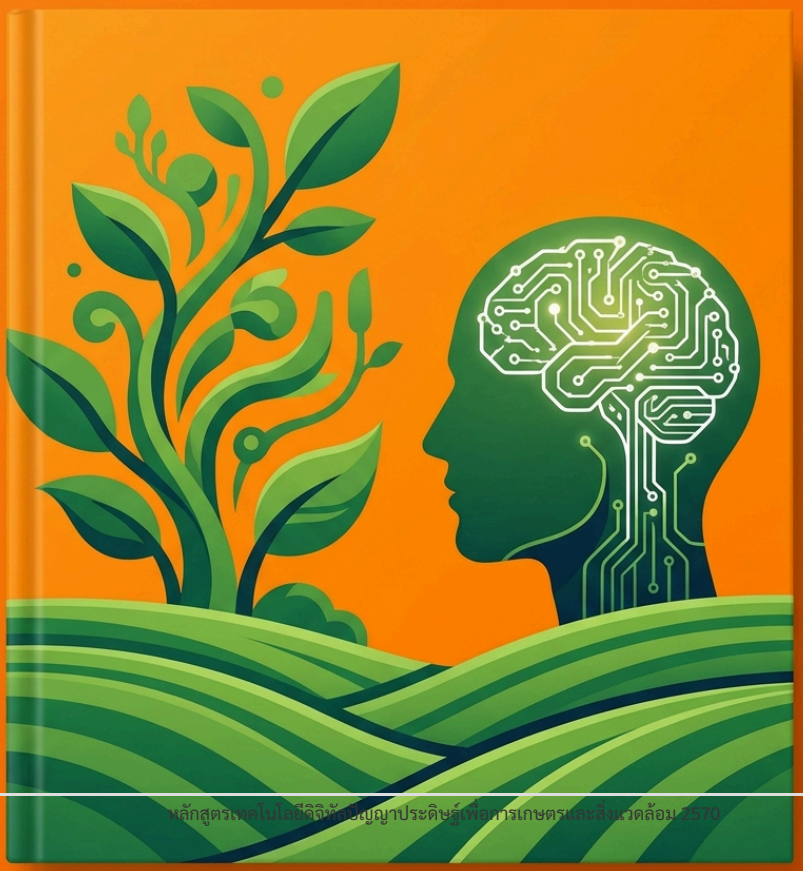
Python xAI

< AI Agent Co-Creator >

Master Instructor Handbook - Facilitator Edition



บทที่ 1: AI Co-Pilot Mastery

ระยะเวลา: ชั่วโมงที่ 1 - 3 (3 ชั่วโมง) **หัวใจสำคัญ**: เปลี่ยนจากการ "เขียนโค้ด" ไปสู่การ "กำกับ AI" เพื่อสร้างตรรกะนวัตกรรม

ชั่วโมงที่ 1: ก้าวแรกสู่ Cloud Lab

ทำความรู้จักกับ Google Colab ห้องทดลองที่ทรงพลังที่สุดบนระบบ Cloud

Copy-Paste Lab: ตรวจสอบระบบ

รันโค้ดชุดนี้เพื่อดูว่า Colab ของคุณพร้อมทำงานหรือไม่

.py / .ipynb

```
import sys
print(f"Python Version: {sys.version}")
print("Cloud Lab Initialized: Ready to Code!")
```

Minecraft Spotlight: พิกัด 3 มิติ (X, Y, Z)

ใน Minecraft โลกประกอบด้วยพิกัด **X (ซ้าย-ขวา)**, **Y (ความสูง)** และ **Z (ลึก)**

- การขยับ Agent ในเกมด้วย Python คือการเปลี่ยนค่า `pos(x, y, z)`
- เช่นเดียวกับอาเรย์ที่เราจะเรียน: `farm[x][y]` คือการระบุตำแหน่งดินในฟาร์ม!

AI Co-Pilot: เพื่อนคู่คิดคนใหม่ (Real-Time Debugging)

ตั้งแต่วินาทีนี้ไป คุณจะไม่ต้องนั่งงมกับ Error คนเดียวอีกต่อไป! หากโค้ดรันไม่ได้ ให้ลอง Copy Error ไปถาม AI ด้วยประโยคนี้:

> *"ฉันเป็นนักเรียนที่กำลังเรียน Python ใน Google Colab และเจอข้อผิดพลาดนี้: [แปะ Error] ช่วยอธิบายว่ามันเกิดจากอะไร และแนะนำวิธีแก้โค้ดทีละขั้นตอนให้หน่อย"*

ชั่วโมงที่ 2: ศิลปะการสั่งงาน AI (S.C.I.E.N.T.I.S.T. Prompting)

เพื่อให้ AI ช่วยคุณสร้างนวัตกรรมที่แม่นยำ เราจะใช้สูตรการสั่งงานระดับโลกที่ครอบคลุมทุกมิติ:

อักษร	ความหมาย	รายละเอียดการสั่ง
S	Situation	บอกสถานะปัจจุบัน ("ฉันกำลังทำโปรเจกต์แยกโรคพืช...")
C	Context	ให้ข้อมูลแวดล้อม ("โดยใช้ข้อมูลอุณหภูมิและความชื้น...")
I	Input	ใส่ข้อมูลหรือโค้ดที่มี ("นี่คือโค้ดที่ฉันมี [แปะโค้ด]...")
E	Expectation	บอกผลลัพธ์ที่อยากได้ ("ฉันอยากให้ AI ตรวจสอบว่าพืชเสี่ยงเป็นโรคไหม...")
N	Nuance	ระบุรายละเอียดปลีกย่อย ("ขอคำอธิบายที่เด็ก ม.ต้น เข้าใจได้...")
T	Target	ระบุกลุ่มเป้าหมายหรือวัตถุประสงค์ ("เพื่อใช้ในการพรีเซนต์โครงงาน...")
I	Insight	ขอคำแนะนำเชิงลึก ("แนะนำแนวทางการพัฒนาต่อยอดให้ด้วย...")

S	Style	กำหนดรูปแบบคำตอบ ("แสดงผลเป็นข้อๆ หรือเป็นตาราง...")
T	Test	ให้ AI ช่วยตรวจสอบ ("ช่วยเขียน Test Case เพื่อเช็คโค้ดนี้ให้หน่อย...")

AI จะฉลาดได้ต้องมี "ตรรกะ" (Logic) เราจะสั่งให้คอมพิวเตอร์ "คิด" และ "ทำ" ตามเงื่อนไข

Copy-Paste Lab: ภารกิจทักทาย (Junior/Senior)

สร้างตัวแปรเพื่อเก็บข้อมูลและแสดงผลลัพธ์

.py / .ipynb

```
# ระบบบันทึกชื่อนักนวัตกรรม
name = input("ระบุชื่อรหัสนักวิทย์ของคุณ: ")
print(f"ยินดีตอนรับคุณ {name} เข้าสู่ปี 2026!")

# ระบบตรวจสอบอุณหภูมิฟาร์มอัจฉริยะ
temp = 38
if temp > 35:
    print("   แจ้งเตือน: อุณหภูมิสูงเกินไป! เปิดพัดลม...")
else:
    print("   สถานะ: อุณหภูมิปกติ")
```

ชั่วโมงที่ 3: ระบบคัดกรองข้อมูล (Data Filtering)

เรียนรู้การทำงานกับข้อมูลหลายๆ ชุด (Lists) และการวนลูป (Loops)

Copy-Paste Lab: เซนเซอร์พยากรณ์

.py / .ipynb

```
# ข้อมูลความชื้นจาก 5 จุดในฟาร์ม
moisture_levels = [45, 32, 55, 28, 40]

for m in moisture_levels:
    status = "ปกติ" if m >= 40 else "ต่ำเกินไป (ต้องรดน้ำ)"
    print(f"จุดตรวจที่ระดับ {m}%: สถานะ {status}")
```

Checklist ท้ายบท

- [ ] รันโค้ดใน Colab สำเร็จ
- [ ] เข้าใจการใช้ `if-else` ในการตัดสินใจ
- [ ] สามารถรัน `for loop` เพื่อวนลูปอ่านค่าข้อมูลได้

Minecraft Challenge: หุ่นยนต์เก็บเกี่ยว (Automated Harvesting)

ลองใช้ความรู้เรื่อง **Loop** ในบทนี้ สั่งให้ Agent เดินเก็บเกี่ยวพืชในรัศมี 10 บล็อกรอบตัว:

.py / .ipynb

```
# สั่งหุ่นยนต์เดินเก็บของในพื้นที่
for length in range(10):
    agent.destroy(FORWARD)
    agent.move(FORWARD, 1)
```

Challenge (คำถามท้าทาย)

หากต้องการให้ระบบแจ้งเตือน "วิกฤต!" เมื่อความชื้นต่ำกว่า **20%** คุณต้องแก้ไขเงื่อนไขในโค้ดบรรทัดไหน?

บทที่ 2: พลังแห่งการพยากรณ์

ระยะเวลา: ชั่วโมงที่ 4 - 6 (3 ชั่วโมง) **หัวใจสำคัญ**: การใช้ Machine Learning ทายอนาคต และอธิบายเหตุผล (xAI)

ชั่วโมงที่ 4: จากข้อมูลสู่ความรู้ (Machine Learning)

เรียนรู้ความแตกต่างระหว่าง "การเขียนโปรแกรมปกติ" และ "การสอน AI"

Copy-Paste Lab: ตรวจสอบความถูกต้องของข้อมูล

ข้อมูลในโลกจริงมักจะมีช่องโหว่ เราต้องทำความสะอาดก่อนเสมอ

.py / .ipynb

```
import pandas as pd
import numpy as np

# จำลองข้อมูลจากฟาร์ม 5 จุด
data = {'ความชื้น': [45, 32, np.nan, 28, 40], 'ปุ๋ย': [10, 15, 12, np.nan, 18]}
df = pd.DataFrame(data)

# ซ่อมข้อมูลแหว่งด้วยค่าเฉลี่ย (Mean)
df.fillna(df.mean(), inplace=True)
print("   ข้อมูลถูกซ่อมเรียบร้อยแล้ว!")
print(df)
```

ชั่วโมงที่ 5: การสอน AI ด้วย Random Forest

เราจะใช้โมเดล "ป่าสุ่มร้อยต้น" มาทำงานแทนเรา

ภาพที่ 4: แดชบอร์ดสรุปประสิทธิภาพของแบบจำลอง Random Forest

Copy-Paste Lab: สร้างและรัน AI Model

.py / .ipynb

```
from sklearn.ensemble import RandomForestClassifier

# 1. ข้อมูลตัวอย่าง [ดินดี, น้ำชุ่ม, แสงพอ]
X = [[1, 1, 1], [0, 0, 0], [1, 0, 1], [0, 1, 0]]
y = ['โตดี', 'เฉา', 'พอใช้', 'พอใช้']

# 2. สร้างโมเดลและฝึกสอน
model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

# 3. ลองให้ AI ทายค่าใหม่!
test_farm = [[1, 1, 0]] # ดินดี, น้ำชุ่ม, แสงน้อย
prediction = model.predict(test_farm)
print(f"   AI ทายว่า: {prediction[0]}")
```

ชั่วโมงที่ 6: เจาะลึกสมอง AI (xAI)

ทำไม AI ถึงตัดสินใจแบบนั้น? เราจะส่องดูค่าน้ำหนักความสำคัญ (Feature Importance)

Copy-Paste Lab: ค้นหาความลับของข้อมูล

.py / .ipynb

```python
# ค้นหาว่าตัวแปรไหนมีผลต่อการตัดสินใจที่สุด
importance = model.feature_importances_
features = ['คุณภาพดิน', 'ปริมาณน้ำ', 'แสงแดด']

for f, imp in zip(features, importance):
    print(f"   {f}: มีความสำคัญ {imp*100:.2f}%")
```

Checklist ท้ายบท

- [ ] สามารถสร้างโมเดล Machine Learning และดูเหตุผล (xAI) ได้
- [ ] เข้าใจการทำงานของ Random Forest
- [ ] สามารถดึงความสำคัญของฟีเจอร์ (Feature Importance) ออกมาได้

Minecraft Challenge: หุ่นยนต์เก็บเกี่ยว (Automated Harvesting)

ลองใช้ความรู้เรื่อง **Loop** ในบทนี้ สั่งให้ Agent เดินเก็บเกี่ยวพืชในรัศมี 10 บล็อกรอบตัว:

.py / .ipynb

```python
# สั่งหุ่นยนต์เดินเก็บของในพื้นที่
for length in range(10):
    agent.destroy(FORWARD)
    agent.move(FORWARD, 1)
```

Challenge (คำถามท้าทาย)

จากผลลัพธ์ของ xAI ปัจจัยไหน (ดิน, น้ำ, หรือแสง) ที่มีน้ำหนักความสำคัญต่อพืชชนิดนี้มากที่สุด?

บทที่ 3: สมองดิจิทัล (Deep Learning)

ระยะเวลา: ชั่วโมงที่ 7 - 8 (2 ชั่วโมง) **หัวใจสำคัญ**: การเลียนแบบเซลล์ประสาทมนุษย์ เพื่อแยกแยะโรคพืช

ชั่วโมงที่ 7: โครงสร้างเซลล์ประสาทเทียม (Neural Networks)

ทำความเข้าใจเลเยอร์ต่างๆ ของสมอง AI (Input, ReLU, Softmax)

Copy-Paste Lab: การเลือกฟังก์ชันกระตุ้น (ReLU)

.py / .ipynb

```
# ReLU คือคนคัดกรอง: ถ้าเป็นค่าลบให้เป็น 0 ถ้าเป็นบวกให้ผ่านได้
sample_values = [-5, 12, -2, 8]
relu_output = [max(0, x) for x in sample_values]
print(f"ผลลัพธ์จาก ReLU สำหรับ {sample_values} คือ {relu_output}")
```

ชั่วโมงที่ 8: ภารกิจสร้างโมเดลแยกโรคพืช (CNN)

เราจะสร้าง AI ที่ "มองเห็น" และแยกโรคพืชได้โดยใช้ภาพ

Copy-Paste Lab: โครงสร้างโมเดลจำแนกภาพ

.py / .ipynb

```
# จำลองการสร้าง AI สมองกล 4 เลเยอร์
def build_agri_brain():
    print("สร้าง Input Layer... (รับภาพใบไม้)")
    print("เพิ่ม ReLU Layer... (กรองข้อมูล)")
    print("เพิ่ม Dropout Layer (0.2)... (ป้องกันการจำผิด)")
    print("เพิ่ม Softmax Layer... (ทำนายว่าเป็นโรคอะไร)")

build_agri_brain()
print("   สมองดิจิทัลพร้อมเรียนรู้แล้ว!")
```

Checklist ท้ายบท

- [ ] เข้าใจหน้าที่ของปุ่ม ReLU ในการกรองข้อมูล
- [ ] เข้าใจการใช้ Dropout เพื่อป้องกันการจำคำตอบ (Overfitting)
- [ ] สามารถอธิบายโครงสร้างพื้นฐานของ Neural Networks ได้

Challenge (คำถามท้าทาย)

หาก AI ทายว่าเป็น "โรคใบไหม้" 80% และ "ปกติ" 20% ฟังก์ชันใดทำหน้าที่สรุปผลตัวเลขเป็นเปอร์เซ็นต์? (Softmax หรือ ReLU)

บทที่ 4: ดวงตาอัจฉริยะและภาษา AI (CV & NLP)

ระยะเวลา: ชั่วโมงที่ 9 - 10 (2 ชั่วโมงเต็ม) **หัวใจสำคัญ**: ก้าวข้ามจาก "ข้อมูล" สู่ "การรับรู้" (Perception) ทั้งภาพและภาษา

ในบทนี้ เราจะกระชับเนื้อหาให้เข้มข้นและรวดเร็ว เพื่อให้นักเรียนสามารถนำ AI ไปประยุกต์ใช้ในการตรวจสุขภาพพืชและสั่งงานด้วยเสียงได้ทันทีครับ

พาร์ทที่ 1: Computer Vision (60 นาที)

แบ่งเป็น 2 ช่วงสั้นๆ เพื่อให้เห็นผลลัพธ์ที่เป็นรูปธรรม

ช่วงที่ 1: ตรรกะระดับพิกเซลและสี (30 นาที)

เรียนรู้การทำงานของ **HSV Masking** ที่แม่นยำกว่า RGB

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex01` *และ* `ex02`

ช่วงที่ 2: การตรวจจับวัตถุด้วยโมเดลสำเร็จรูป (30 นาที)

การใช้ **Haar Cascades** เพื่อตรวจหาใบพืชหรือวัตถุในฟาร์ม

.py / .ipynb

```python
# ตัวอย่างการโหลดโมเดล (Boilerplate)
import cv2
face_cascade = cv2.CascadeClassifier('models/haarcascade_agri.xml')
# ... ตรวจจับและวาดกรอบสี่เหลี่ยม ...
```

พาร์ทที่ 2: Natural Language Processing (60 นาที)

เปลี่ยนข้อความเป็นคำสั่งงานเชิงลึก

ช่วงที่ 3: Intent & Sentiment (30 นาที)

เข้าใจว่า AI "เดาใจ" ผู้ใช้อย่างไร ผ่านคีย์เวิร์ดและคะแนนความรู้สึก (Sentiment)

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex03` *และ* `ex04`

ช่วงที่ 4: Agri-Bot Liaison (30 นาที)

ประกอบร่างเป็นระบบแชทบอทถาม-ตอบเรื่องการเกษตรที่โต้ตอบได้จริง

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex05`

Checklist ท้ายบท

- [ ] สามารถแยกเฉดสีน้ำตาล (โรคพืช) ออกจากใบไม้ได้สำเร็จ
- [ ] เข้าใจการทำงานของระบบตัดสินใจผ่าน Intents
- [ ] สามารถประยุกต์ใช้โมเดลสำเร็จรูป (Inference) ในการตรวจจับวัตถุได้

> *[!IMPORTANT] **คำแนะนำสำหรับวิทยากร**: หากเวลาเหลือน้อย ให้เน้นที่การรันโค้ดสำเร็จรูปใน `ta_solutions` เพื่อให้เด็กๆ ได้เห็นผลลัพธ์ที่น่าตื่นเต้น (Wow Factor) ก่อนจบวัน*

บทที่ 5: Digital Twin Simulation (2D & 3D)

ระยะเวลา: ชั่วโมงที่ 11 (1 ชั่วโมง) **หัวใจสำคัญ**: การเชื่อมโยงโลกเสมือน (Software) เข้ากับตรรกะ AI เพื่อจำลองการทำงานก่อนลงพื้นที่จริง

ในบทนี้ เราจะเปลี่ยนจากการ "เขียนเกม" ทั่วไป มาเป็นการสร้าง **Digital Twin Dashboard** ซึ่งเป็นเทคโนโลยีที่นักวิทยาศาสตร์ใช้จำลองฟาร์มหรือโรงงานในคอมพิวเตอร์ เพื่อทดสอบโค้ด AI ของเราว่าทำงานได้ถูกต้องไหม ก่อนจะเอาไปใส่ในหุ่นยนต์จริงๆ ครับ

ตอนที่ 1: 2D Dashboard ด้วย Pygame Zero

การเริ่มต้นเขียนโปรแกรมจำลองด้วย **Pygame Zero (PyZero)** นั้นง่ายและเห็นผลเร็ว เหมาะสำหรับสร้างหน้านักแสดง (Dashboard) เพื่อดูสถานะของฟาร์ม

โครงสร้างเวทมนตร์ของ PyZero (3 Magic Parts)

1. **Setup**: ตั้งค่าหน้าจอ
2. **Draw**: วาดภาพลงบนจอ (เหมือนพู่กันวิเศษ)
3. **Update**: ตรรกะการขยับและความคิด (Logic)

Copy-Paste Lab: Agri-Monitoring System

ลองคัดลอกโค้ดนี้ไปรัน เพื่อสร้างระบบติดตามรถเก็บเกี่ยวอัตโนมัติ

.py / .ipynb

```
import pgzrun

# 1. ตั้งค่าหน้าจอ
WIDTH = 800
HEIGHT = 600

# สร้างตัวละคร (อย่าลืมใส่รูป harvester.png ในโฟลเดอร์ images)
harvester = Actor('harvester')
harvester.pos = 400, 300

# 2. ส่วนของการวาดผลลัพธ์
def draw():
    screen.clear()
    screen.fill((34, 139, 34)) # สีเขียวฟาร์ม
    harvester.draw()
    screen.draw.text("Agri-AI Simulation Mode", (20, 20), color="white", fontsize=30)

# 3. ตรรกะการควบคุม (AI Logic Simulation)
def update():
    # จำลองว่าถ้า AI ตรวจพบพืชพร้อมเก็บ (กด space) ให้รถขยับ
    if keyboard.space:
        harvester.x += 2

pgzrun.go()
```

ตอนที่ 2: 3D Digital Twin ด้วย Minecraft Education

เมื่อเข้าใจตรรกะใน 2 มิติแล้ว เราจะอัปเกรดเข้าสู่โลก 3 มิติ เพื่อทดสอบว่า "โค้ด" ของเราสามารถสั่งงานหุ่นยนต์ (Agent) ให้ทำงานในพื้นที่จริงได้อย่างไร

เริ่มต้นใช้งาน Code Builder

1. เปิดเกม **Minecraft Education**

2. กดปุ่ม **"C"** เพื่อเปิด Code Builder
3. เลือกภาษา **"Python"**

Lab: หุ่นยนต์เสกไร่ข้าว (The Instant Farm)

Mission: ใช้ Loop เพื่อสั่งให้ Agent ปลูกข้าว 10x10 บล็อก

.py / .ipynb

```
# สั่งให้หุ่นยนต์เดินและปลูกพืชอัตโนมัติ (Nested Loop)
for i in range(10):
    for j in range(10):
        agent.place(FORWARD, 1) # วางบล็อก
        agent.move(FORWARD, 1) # เดินหน้า
    agent.move(LEFT, 1) # ขยับแถวใหม่
    agent.turn(LEFT)
```

Multimodal Logic: เชื่อมโยงความรู้

ในฐานะ **AI-Next Scientist** คุณสามารถรวมทุกบทที่เรียนมาเข้าด้วยกัน:

1. **บทที่ 2-3 (Brain)**: ใช้ ML ทายว่าฝนจะตกไหม
2. **บทที่ 4 (Vision)**: ใช้ CV มองเห็นพื้นที่ใน Minecraft
3. **บทที่ 5 (Body)**: สั่งให้ Agent ขยับตามคำสั่งที่ Brain และ Vision ประมวลผลได้!

Checklist ท้ายบท

- [ ] เข้าใจความหมายของ Digital Twin และการจำลอง (Simulation)
- [ ] สามารถขยับตัวละครใน 2D ได้สำเร็จ
- [ ] สามารถสั่ง Agent ใน Minecraft ให้ทำงานตามตรรกะ Loop ได้

> *[!IMPORTANT] **พี่ยักษ์สรุปให้**: การจำลองในโลก Digital ช่วยลดความเสี่ยงและความผิดพลาดในโลกจริง (Physics World) นี่คือพลังของนักวิทยาศาสตร์ยุค 2026 ครับ!*

บทที่ 6: Agri-Hackathon Stage

ระยะเวลา: ชั่วโมงที่ 12 (1 ชั่วโมง) **หัวใจสำคัญ**: การบูรณาการความรู้ทั้งหมดและนำเสนอผลงาน

ชั่วโมงที่ 12: การสร้างนวัตกรรมต้นแบบและนำเสนอผลงาน

รวบรวมทุกชิ้นส่วน (Logic + ML + Vision + NLP) มาแก้ปัญหาคนเกษตร

Copy-Paste Lab: โครงสร้างโปรเจกต์ต้นแบบ

.py / .ipynb

```python
# ระบบ AI ควบคุมฟาร์มอัจฉริยะแบบเบ็ดเสร็จ
def agri_startup_logic(sensor_input, ai_prediction):
    if sensor_input > 40:
        return "ACTIVATE_COOLING"

    if ai_prediction == 'READY':
        return "DEPLOY_HARVESTER"

    return "MONITORING_MODE"

print(f"   แผนธุรกิจของคุณ: {agri_startup_logic(42, 'READY')}")
```

(ต่อ) การนำเสนอผลงาน (The 3-Minute Pitch)

เล่าเรื่องราวความสำเร็จของคุณให้น่าสนใจ!

โครงสร้างการเล่าเรื่อง (Pitch Template)

1. **The Pain (ปัญหา)**: "รู้ไหมครับ... เกษตรกรแถวบ้านผมต้องเจอกับปัญหาเพลี้ยลงทุกปี..."
2. **The AI Solution (การแก้ปัญหา)**: "พวกเราจึงสร้าง AI ที่มองเห็นเพลี้ยผ่านกล้อง (CV) และทายเวลาเพลี้ยระบาด (ML)..."
3. **The Impact (ผลลัพธ์)**: "นวัตกรรมนี้จะลดการใช้สารเคมีได้ 40% และเพิ่มรายได้ 3 เท่า!"

Checklist ท้ายบท

- [ ] บูรณาการ Logic และ AI ได้เรียบร้อย
- [ ] มีผลลัพธ์การรันจาก Colab ที่แสดงผลความถูกต้อง
- [ ] เตรียมสไลด์นำเสนอ (Pitch) เสร็จทันเวลา

ยินดีด้วยคุณคือ Next-Gen Scientist 2026!

คุณได้ก้าวข้ามขีดจำกัดจากผู้ใช้ มาเป็น **"ผู้สร้าง"** เรียบร้อยแล้ว ขอให้นำความรู้นี้ไปต่อยอดเพื่อสร้างสรรค์โลกที่น่าอยู่ขึ้นด้วย AI นะครับ!